

The Best of Both Worlds: Hybrid Authenticated Key Exchange for QKD(N) without Signatures

Sebastian Clermont¹ and Johanna Henrich²

¹ Technical University of Darmstadt, Darmstadt, Germany

² Darmstadt University of Applied Sciences, Darmstadt, Germany
sebastian.clermont@tu-darmstadt.de, johanna.henrich@h-da.de

Abstract. Post-Quantum Cryptography (PQC) and Quantum Key Distribution (QKD) are both contenders for securing communication against quantum adversaries, but are at different stages of maturity. For hedging security risks, hybridization is the default approach. Unlike previous research on classical–post-quantum hybrids, we propose a QKD-PQC hybrid for Authenticated Key Exchange (AKE).

To minimize the attack surface, we completely remove the requirement for digital signature schemes and propose a Hybrid Authenticated Key Exchange (HAKE) that combines Post-Quantum (PQ) AKE and QKD key agreement, leveraging Key Encapsulation Mechanisms (KEMs) for both key exchange and authentication. Our fully modular security analysis, based on the recent multi-input Key Derivation Function (KDF) framework by Backendal et al. (Eurocrypt 2025), establishes AKE security in the CK01 model and yields a conditional information-theoretic security guarantee when the QKD component is uncompromised; a property not achieved by prior hybrid protocols. We demonstrate the protocol’s practical feasibility with benchmarks using ML-KEM, FrodoKEM, and Classic McEliece.

Keywords: Hybrid Authenticated Key Exchange · Key Combiner · Quantum Key Distribution · Post-Quantum Cryptography · Key Encapsulation Mechanism · Provable Security · Benchmark

1 Introduction

Public-key cryptography encompasses mechanisms such as key exchange and digital signatures that enable secure communication. One such application is

* This work was funded by the German Federal Ministry of Research, Technology and Space and the Hessian Ministry of Higher Education, Research, Science and the Arts within their joint support of the National Research Center for Applied Cybersecurity ATHENE. We would like to thank Fabian Seidl, former research associate and founder of conduit, for his valuable support in implementing the benchmarking framework and for his technical guidance throughout this aspect of the work. Further, we gratefully acknowledge Robin Böhm and Oleg Nikiforov from Deutsche Telekom for valuable discussions on the QKDN integration.

Authenticated Key Exchange (AKE), where two parties securely negotiate a shared secret. Such protocols often use two distinct public-key schemes for key establishment and entity authentication, with KEMs and digital signatures being the most common combination.

However, sufficiently powerful quantum computers (QCs), which may become available within the next few years [18], are capable of breaking classical asymmetric algorithms such as Elliptic-curve Diffie–Hellman (ECDH) and RSA in polynomial time [42]. Moreover, harvest-then-decrypt attacks [40] pose a threat to today’s encrypted communication, thereby enabling the recording of data for later decryption.

Besides PQC [44], which is based on mathematical problems, QKD uses quantum effects to establish pairwise shared secrets in an information-theoretically secure way [33]. Due to the quantum phenomena used for key establishment, traditional attacks such as the aforementioned harvest-then-decrypt attacks are not applicable to QKD. Furthermore, QKD is the only current technology that enables the exchange of truly random keys and thus supports information-theoretic secure (ITS) communication. In addition to these advantages, QKD faces several challenges that must be overcome before it is ready for widespread adoption. Currently, there are no reliable, market-ready QKD modules that can bridge distances of more than 150 km [33]. Moreover, it is only possible to implement point-to-point connections and star topologies without additional measures. That is why QKD modules are often concatenated into QKD Network (QKDN) to distribute a key using hop-by-hop forwarding. This topology makes such a network vulnerable to insider attacks by compromised nodes. In addition, security vulnerabilities during implementation, such as insufficient authentication measures or side-channel leaks, must be given due consideration [18].

Hybridization can hedge security risks by combining the security guarantees of different schemes, ensuring that the security objectives are met as long as at least one of the schemes used has not been compromised [22]. Therefore, a HAKE based on both PQC and QKD can establish a secure channel that withstands an attacker, provided that at least one of these technologies remains secure. This approach aligns with the recommendations [20] of various European security organizations, including Federal Office for Information Security (BSI), French Cybersecurity Agency (ANSSI), and Netherlands National Communications Security Agency (NLNCSA). They conclude that QKD alone currently poses various risks from both theoretical and practical perspectives and should be used only as a supplementary security component.

1.1 Related Work

Entity authentication without signatures, e.g., via KEM-based authentication, is explored in several papers. Schwabe et al. describe a variant of TLS 1.3 that replaces the handshake’s signature procedures with KEM-based authentication [41]. In addition, other AKE protocols with KEM-based authentication are discussed in the literature [25,21].

Boyd et al. provide modular building blocks to generate different AKE protocols that are suitable for instantiation with PQC [8]. In essence, this process provides various authentication options, both with and without signatures. The constructions are generic, allowing for instantiation with suitable primitives.

With respect to HAKE, Bindel et al. describe the general modeling of a HAKE using KEMs [7]. Dowling et al. describe the HAKE protocol Muckle [16], which combines classical schemes with PQC, QKD, and symmetric Pre-Shared Key (PSK). Authentication is done using a PSK, which is unfeasible for general deployments because preshared secrets must be established out of band and be available for each distinct pair of communicating parties. Bruckner et al. extend Muckle and call the result the Muckle+ protocol [9]. Muckle+ allows for PQC+QKD hybrid AKE using digital signatures. Using a Public Key Infrastructure (PKI) enables a hierarchical trust model, as is common practice. In a supplementary extension, Muckle# is also described, which provides KEM-based authentication [3]. However, as in Muckle+, it is assumed that QKD keys are synchronized internally and can be retrieved without specifying an identifier, whereas the industry standard [38] requires key retrieval via explicit key IDs. Moreover, Muckle# does not employ a formally analyzed key combiner and does not achieve the conditional information-theoretic security guarantee that our construction provides (cf. Theorem 2).

Ricci et al. describe a HAKE combining classical and PQC schemes with QKD using KEM-based authentication [39], but focus on benchmarking rather than formal security analysis of the key combiner.

Hövelmanns et al. propose a QKD + PQC hybrid protocol proven secure in a variant of the CK+ model [28] with modifications to accommodate QKD [24]. They identify misunderstandings on using key combiners in other designs but use an XOR-based combiner that introduces a dependent-key vulnerability, fixable by using a combiner that binds context information to the final key [1].

1.2 Our Contributions

We employ a combination of QKD and PQC to design a HAKE that operates entirely without digital signature schemes or symmetric PSKs during protocol execution, thereby reducing system complexity and overall attack surface. This choice eliminates the need to deploy a separate signature-key infrastructure and avoids the substantial communication and computational overhead of post-quantum digital signatures, which can inflate handshake sizes by several kilobytes per operation. Our construction integrates the KEM-based AKE of Boyd et al. [8] with a QKD key agreement and applies the multi-input KDF framework of Backendal et al. [1] to obtain security guarantees that neither component achieves alone. Its security is established in the well-known CK01 model [10], which we choose for compatibility with the underlying KEM-based AKE and because it already captures forward secrecy. More recent models such as eCK provide additional guarantees around ephemeral key leakage, but these are less relevant in the KEM-based setting where ephemeral secrets are not long-lived Diffie–Hellman exponents.

This yields a fully modular security analysis: the two component protocols are treated in isolation, and hybrid security follows from the combiner. If either component is replaced or improved, only the corresponding bound needs updating, without re-proving the entire protocol.

Crucially, this modular analysis yields a *conditional information-theoretic security* guarantee (Theorem 2): when the QKD system is uncompromised, the resulting session key is information-theoretically secure, even if all computational assumptions are broken. This property strictly exceeds what prior PQC–QKD hybrid protocols achieve. Prior constructions derive the combined key via hash-based key derivation, which yields computational rather than information-theoretic security even when one component provides unconditionally secure keys. This provides a graceful degradation guarantee: when QKD is secure, the resulting session key is information-theoretically secure; should QKD fail, the session key retains its full computational security under standard assumptions.

Finally, we have implemented the protocol with various KEMs and provide benchmarks on the real-world performance.

2 Preliminaries

2.1 Notation

This paper uses established notation for cryptographic papers. λ denotes the security parameter. $a \leftarrow b$ denotes deterministic assignment of the value b to variables a or assigning the output of the deterministic algorithm b to a . $c \leftarrow \$ d$ denotes assigning the output of the probabilistic algorithm d to the variable c . $\llbracket \text{expr} \rrbracket$ denotes the boolean evaluation of the expression expr . $=$ denotes an equality check. The symbol $\perp$ indicates failure, $:=$ denotes a definition. $\parallel$ denotes the concatenation of values, $\Pr[\text{event}]$ the probability of some event and $|\text{value}|$ the absolute value. All PPT and QPT adversaries are efficient adversaries, which means their runtime and memory are upper-bounded by some polynomial that is dependent on λ . For the advantage of adversaries, we write $\text{Adv}_{\text{scheme}, \mathcal{A}}^{\text{property}}(\lambda)$ for the advantage of an adversary $\mathcal{A}$ against the security property **property** of some cryptographic scheme *scheme*. $\text{Exp}_{\text{scheme}, \mathcal{A}}^{\text{property}}(\lambda)$ is used to define the corresponding experiment. Defining security of KDFs requires additional parameters; these advantage terms are introduced in Section 2.4.

For oracle definitions, we write “ $\text{O}_{\text{Decaps}}(\text{sk}, \cdot)$ on input c ” for an oracle called **Decaps**. The oracle takes a single input, as indicated by the single “ $\cdot$ ”, and we will call this input “ c ” in the algorithmic description of the oracle. We make sk explicit in the oracle description to highlight which secrets an adversary is given indirect access to.

2.2 Key Encapsulation Mechanisms

A KEM is an asymmetric primitive that can be used to establish a shared secret between two parties [13]. It consists of three algorithms.

- $(pk, sk) \leftarrow \$ KGen(1^\lambda)$: The probabilistic algorithm that, given the security parameter λ , generates a key pair (pk, sk) .
- $(ct, k) \leftarrow \$ Encaps(pk)$: **Encaps** is a probabilistic algorithm, taking the pk to compute a ciphertext ct encapsulating a shared secret and the shared secret k itself.
- $k \leftarrow Decaps(sk, ct)$: **Decaps** is a deterministic algorithm, using sk and ct to obtain the corresponding k .

We call a KEM ε -correct, iff for all $(sk, pk) \leftarrow \$ KGen(1^\lambda)$ it holds that if $(ct, k) \leftarrow \$ Encaps(pk)$ and $k' \leftarrow Decaps(sk, ct)$, then $k' = k$ except with negligible probability.

$Exp_{KEM, \mathcal{A}}^{IND-CPA}(\lambda)$:	$Exp_{KEM, \mathcal{A}}^{IND-CCA}(\lambda)$:	$O_{Decaps}(sk, \cdot)$ on input c :
1 $(pk, sk) \leftarrow \$ KGen(1^\lambda)$	7 $(pk, sk) \leftarrow \$ KGen(1^\lambda)$	13 if $\llbracket c = c^* \rrbracket$
2 $k_0 \leftarrow \$ \mathcal{K}$	8 $k_0 \leftarrow \$ \mathcal{K}$	14 return $\perp$
3 $(c^*, k_1) \leftarrow \$ Encaps(pk)$	9 $(c^*, k_1) \leftarrow \$ Encaps(pk)$	15
4 $b \leftarrow \$ \{0, 1\}$	10 $b \leftarrow \$ \{0, 1\}$	return $Decaps(sk, c)$
5 $b' \leftarrow \$ \mathcal{A}(pk, c^*, k_b)$	11 $b' \leftarrow \$ \mathcal{A}^{O_{Decaps}}(pk, c^*, k_b)$	
6 return $\llbracket b' = b \rrbracket$	12 return $\llbracket b' = b \rrbracket$	

Fig. 1. Game definition for IND-ATK security of key encapsulation mechanisms with $ATK \in \{CPA, CCA\}$.

Definition 1 (IND-ATK security of KEM schemes). *Given the security game in Figure 1, we define the advantage of an adversary against IND-ATK security of a key encapsulation scheme KEM for $ATK \in \{CPA, CCA\}$, as*

$$Adv_{KEM, \mathcal{A}}^{IND-ATK}(\lambda) := \left| \Pr \left[Exp_{KEM, \mathcal{A}}^{IND-ATK}(\lambda) = 1 \right] - \frac{1}{2} \right|.$$

2.3 Message Authentication Codes

A Message Authentication Code (MAC) is a symmetric scheme that can be used to ensure integrity and authenticity of messages. A MAC scheme comprises three algorithms.

1. $k \leftarrow \$ KGen(\lambda)$: The probabilistic algorithm **KGen** takes the security parameter λ as input and generates a key k .
2. $\tau \leftarrow \$ Tag(k, m)$: The probabilistic algorithm **Tag** maps a key k and a message $m \in \{0, 1\}^*$ to a tag τ .
3. $d \leftarrow Vrfy(k, m, \tau)$: The deterministic algorithm **Vrfy** takes a key k , a message m and a tag τ and outputs a decision $d \in \{\text{true}, \text{false}\}$, indicating if the tag is valid for given message and key.

The algorithms operate over key-, message-, and tag-spaces, which we refer to as $\mathcal{K}, \mathcal{M}, \mathcal{T}$, respectively.

We call a MAC scheme perfectly correct, iff

$$\text{Vrfy}(k, m, \text{Tag}(k, m)) = \text{true}$$

for any $k \leftarrow \$ \text{KGen}(\lambda)$ and $m \in \mathcal{M}$.

$\text{Exp}_{\text{Mac}, \mathcal{A}}^{\text{EUF-CMA}}(\lambda):$	$\text{O}_{\text{Tag}}(k, \cdot)$ on input m :
1 $k \leftarrow \$ \text{KGen}(\lambda)$	5 $\tau \leftarrow \$ \text{Tag}(k, m)$
2 $\mathcal{L}_m \leftarrow \emptyset$	6 $\mathcal{L}_m \leftarrow \mathcal{L}_m \cup \{m\}$
3 $(m, \tau) \leftarrow \$ \mathcal{A}^{\text{O}_{\text{Tag}}}$	7 return τ
4 return $\llbracket \text{Vrfy}(k, m, \tau) \wedge m \notin \mathcal{L}_m \rrbracket$	

Fig. 2. Game definition for EUF-CMA security of MAC schemes.

Definition 2 (EUF-CMA security of MAC schemes). *Given the security game in Figure 2, we define the advantage of an adversary $\mathcal{A}$ against the EUF-CMA security of a MAC scheme MAC as:*

$$\text{Adv}_{\text{MAC}, \mathcal{A}}^{\text{EUF-CMA}}(\lambda) := \Pr [\text{Exp}_{\text{MAC}}^{\text{EUF-CMA}}(\mathcal{A}) = 1].$$

2.4 Key Derivation Functions

For analyzing the security of multi-input KDFs (n -KDFs), as we use in the constructions we will introduce later, we use the newly established notion by Backendal et al. [1].

An n -KDF is a deterministic algorithm, which we also refer to as a *key combiner*, that takes up to n input pairs $(\sigma_1, c_1), \dots, (\sigma_n, c_n)$, a protocol-specific label lbl , and a desired output length L , producing a combined key K :

$$K \leftarrow F((\sigma_1, c_1), \dots, (\sigma_n, c_n), \text{lbl}, L).$$

The secret key material σ_i is the secret on which the n -KDF is evaluated, typically a shared secret established by a key exchange protocol. c_i denotes context information relevant to the creation of the key material input (e.g., in a Diffie–Hellman style key exchange, g^{xy} is the secret σ and a description of the underlying algebraic group together with the shares g^x, g^y is the context c). The label lbl is a fixed string that identifies the protocol and purpose of the derived key, providing domain separation and preventing cross-protocol attacks.

The inputs (σ_i, c_i) are sampled from so-called *key material sources*, denoted by Σ , which can be of different types. E.g., keys obtained from a KEM-style key agreement protocol are sampled from some source Σ_{KEM} , while keys obtained by a QKD system are sampled from Σ_{QKD} . The set of sources used by a KDF is

denoted as the source collection Σ . The mapping from key material sources to the input position of the KDF is fixed and encoded in a variable called $\Sigma\text{-map}$. The combiner function F used in the protocol presented in the next section expects a key (and associated context) from Σ_{KEM} as its first input, and Σ_{QKD} as its second input.

Security is defined using a real-or-random indistinguishability game that can be customized using a predicate req , which will be req_{Hybrid} throughout this paper. This predicate, as introduced by Backendal et al., intuitively requires that one of the sources of key material (i.e., either Σ_{KEM} or Σ_{QKD}) consistently outputs “good”, i.e. keys that are not known by an adversary. In the indistinguishability game, an adversary has the ability to run the n -KDF honestly with arbitrary inputs and has access to a challenge oracle that evaluates either the real n -KDF or outputs a uniformly random key for new inputs. We write the advantage of an adversary against the KDF security of F with sources Σ and predicate req as $\text{Adv}_{\mathcal{A}, \Sigma, req}^{\text{kdf}}(\lambda)$.

The security bounds demonstrated by Backendal et al. that are relevant for this work are based on the unpredictability of a key material source Σ . Intuitively, this means that, given all observable information, an adversary is unable to predict the keys obtained from this key exchange. This effectively reduces proving the security of the combiner to proving that the key exchange protocol generates random keys, a notion captured by the SK security discussed in Section 2.5. We write $\text{Adv}_{\mathcal{A}, \Sigma}^{\text{up}}(\lambda)$ to denote the advantage of an adversary $\mathcal{A}$ against the unpredictability of source Σ . This unpredictability of the session keys is a weaker notion than the pseudorandomness obtained from key exchanges shown to be secure in the CK01 model [1, Lemma 23].

We choose this security model for our analysis because it explicitly allows leakage of some input secrets. This could be caused by a quantum computer breaking classical assumptions, or by a faulty implementation of Quantum Key Distribution, which is exactly the setting we are interested in.

The concrete instantiation chosen for our implementation and used during benchmarks is the “Random-Oracle-Based Construction” [1, Figure 4], instantiated with SHA-3-256 [17].

2.5 Authenticated Key Exchange

Since our work builds on the work by Boyd et al. [8], our results are in the same security model for authenticated key exchange, which, in turn, adopts the work by Canetti and Krawczyk [10], often referred to as the CK01-model. We adopt their notion of SK-secure AKE algorithms and call a protocol Π a secure authenticated key exchange, if it is SK-secure and provides PFS (formally defined in the PFS paragraph below). We will now formally introduce the required definitions.

The CK01 Setting: In the CK01 model, a key exchange is a message-driven protocol executed between two parties. Multiple instances of a protocol can be run in parallel between parties. We call each singular run a session and characterize it by the tuple $(P_i, P_j, s, \text{role})$, where P_i intends to communicate with P_j in

a session with the identifier s and P_i has the role $\text{role} \in \{\text{initiator}, \text{responder}\}$. When a party completes its session, it computes its local output, which, most importantly, includes the session key.

Definition 3. Matching Sessions *We call two sessions matching, when for a session at P_i of the form $(P_i, P_j, s, \text{initiator})$ a corresponding session at P_j of the form $(P_j, P_i, s', \text{responder})$ exists and $s = s'$.*

Selecting adequate session identifiers is outside the scope of the model. We follow Krawczyk et al.'s assumption that there is some higher-level communication protocol ensuring the setup of unique session identifiers, e.g., by using a concatenation of random values generated by each party [10].

The model is initialized during a setup phase, where all participating parties generate their long-term keys without any influence from the adversary and make them available to all potential communication partners. Only after completing this initialization step can the adversary start interacting with the parties.

SK-secure key exchange: We now discuss what the CK01 model requires for AKE protocols to be secure, stating the adversary capabilities followed by the security definition. For the full formalization, we refer to [10,8].

In the CK01 model, an adversary has full control over all communication: it can withhold, fabricate, misdirect, or faithfully deliver messages, and can initiate protocol runs between parties. This complete control includes the ability to observe all messages sent as part of the protocol (which might be encrypted).

In addition to controlling the message flow, the adversary has access to oracles that reveal secrets, modeling partial corruptions in the system. Specifically, the adversary has access to three oracles

- Corrupt Oracle. This allows an adversary to learn a party's current internal state, including all its local secrets.
- Session-State Oracle. This allows an adversary to learn the internal state of a given session at a given party. The affected session must not have completed (i.e., derived the shared key) when the query is issued.
- Reveal Oracle. This allows an adversary to reveal the session key for a completed session.

Then, there is a **Test Oracle**. The adversary can query the **Test** oracle for any completed (i.e. calculated a session key) and fresh (i.e. the adversary cannot trivially obtain the session key) session (P_i, P_j, s) . Depending on a bit b , the oracle will output either the actual session key or a uniform bitstring of the same length.

To model forward secrecy, a fifth oracle is added to the model.

- Expire-Session Oracle. This oracle expires a session, which deletes the derived session key at the respective parties.

Definition 4 (Freshness (informal)). *A session $(P_i, P_j, s, \text{role})$ is fresh (and can thus be tested by an adversary), if the following conditions hold:*

1. The session key has not been revealed.
2. The session state has not been revealed.
3. If it has been corrupted, it must have been expired first.
4. The same holds for the matching session $(P_j, P_i, s, \text{role}')$.

Observe that our definition of freshness includes forward secrecy.

Perfect Forward Secrecy Perfect forward secrecy (PFS) guarantees that previously established session keys remain secure even if long-term keys are later compromised. We model PFS in the CK01 model using the Expire-Session Oracle. Namely, a session is still considered fresh (and can therefore be tested by the adversary), even if it has been corrupted before, as long as the session has been expired before the corruption.

Definition 5 (Key-indistinguishability security (informal)). *The security of an AKE protocol Π is defined using the key indistinguishability experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{Key-IND}}(\lambda)$. The adversary has the aforementioned capabilities and oracles with the objective of successfully identifying a real key from a random key without violating any freshness conditions. The advantage of $\mathcal{A}$ is defined as*

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{Key-IND}}(\lambda) := \left| \Pr \left[\text{Exp}_{\Pi, \mathcal{A}}^{\text{Key-IND}}(\lambda) = 1 \right] - \frac{1}{2} \right|.$$

Definition 6 (SK-Security). *A key exchange protocol Π is SK-secure, if the following two conditions hold:*

- (1) *If two uncompromised parties complete matching sessions of the protocol Π , they obtain the same session key.*
- (2) *Any efficient adversary $\mathcal{A}$ attacking key-indistinguishability of protocol Π has at most a negligible advantage over randomly guessing.*

We extend this definition to the PQ setting by requiring indistinguishability from random for a quantum polynomial time (QPT) adversary. Since the parties performing the key exchange use classical hardware, we do not grant the adversary superposition access to any of the key exchange oracles. For the QKD protocol, the central entity is the detector that measures the incoming particles, which we also do not need to model with a quantum oracle.

To prevent trivial wins, Definition 6 requires the adversary to test a fresh session. Informally, a session is fresh if the adversary does not trivially know the session key, i.e. by corrupting both parties involved in a key exchange or revealing the secret of the tested session.

2.6 QKD as a special case of AKE

Quantum Key Distribution (QKD) is a key establishment protocol whose security relies on fundamental physical principles rather than computational hardness assumptions, dating back to the seminal work of Bennett and Brassard [4].

Despite a long research history, challenges remain for mainstream adoption, including limited range (roughly 150 km) and low key rates in the neighborhood of single-digit kbit/s. While acknowledging these limitations, European cybersecurity agencies agree that research must continue [20].

Due to the nature of QKD, functionality is provided by specialized hardware devices (QKD nodes) with physical connections to neighboring nodes. These nodes continuously negotiate key material, which is made available to clients via an interface standardized by the European Telecommunications Standards Institute (ETSI) in GS QKD 014 [38]. A Key Management System (KMS) is typically situated above the QKD node to manage keys and facilitate user access. Communication partners obtain a QKD key from their respective nodes using a $qkdKeyId$, assigned uniformly at random by the QKD node.

Definition 7 (QKD Interface). *Two parties connected to neighboring QKD nodes have the following functionality available to establish shared keys:*

- $(qkdKeyId, k_{qkd}) \leftarrow \text{GetKey}$: *When the algorithm GetKey is called with no input, it selects an available QKD key and returns it along with the corresponding unique identifier.*
- $k_{qkd}/\perp \leftarrow \text{GetKeyById}(qkdKeyId)$: *The algorithm has to be executed at the partnered node, where the corresponding key will be retrieved, or an error will be indicated by outputting the dedicated failure symbol $\perp$.*

Each key can be output at most once; repeated queries to GetKeyById with a previously used $qkdKeyId$ return $\perp$.

In order to establish a shared key using QKD, the initiating party retrieves a QKD key and an associated identifier $qkdKeyId$ from its QKD node using GetKey . It then transmits $qkdKeyId$ to the intended partner, who uses $\text{GetKeyById}(qkdKeyId)$ to retrieve the same key from its own QKD node, thereby finalizing the key exchange. We call this protocol Π_{QKD} .

For this paper, we view QKD as a black box that delivers truly random keys as promised, including the classical communication over an authenticated channel. Established protocols include BB84 [4], Ekert91 [19], and BBM92 [5], which rely on two orthogonal quantum states, entanglement in combination with Bell inequalities, and entanglement with complementary bases, respectively.

Unlike other approaches such as [24,9] we do not model QKD with all its intricate details. This black-box abstraction is justified for two reasons. First, it preserves generality: modeling protocol-specific details (detector characteristics, authentication of the classical channel) would tie the analysis to a particular QKD implementation, whereas our construction is generic. Second, it is consistent with the established approach in the QKD security literature, where unconditional security proofs [32] yield a composable guarantee that the output is indistinguishable from ideal random keys, up to a failure probability. We collect all possible deviations from ideal behavior, including implementation vulnerabilities and side-channel leaks, under a single umbrella event QKD_fail . The concrete probability $\Pr[\text{QKD_fail}]$ can be instantiated from the security proof

of the specific QKD protocol deployed, or set conservatively based on the risk tolerance of the deploying entity.

Lemma 1. Π_{QKD} as described above is SK-secure (which in our definitions includes PFS) against any unbounded adversary $\mathcal{A}$. It holds that

$$\text{Adv}_{\Pi_{\text{QKD}}, \mathcal{A}}^{\text{SK-secure}}(\lambda) \leq \Pr[\text{QKD_fail}], \quad (1)$$

where the event QKD_fail indicates that the underlying QKD system failed in any way, e.g., by not returning identical keys for identical key identifiers, the output key not being uniformly random, or even just a constant value.

This notion combines correctness (identical keys for identical identifiers) and security (here: SK security) into one convenient term.

Proof. While this claim has been analyzed with much greater formal rigor (but similar results) than here, e.g., in [32], this proof will show how to achieve the stated bound when assuming QKD to work as a black box.

First, we show that black-box QKD achieves key indistinguishability via a series of game hops.

Game₀(λ): We start with the regular key indistinguishability game $\text{Exp}_{\Pi_{\text{QKD}}, \mathcal{A}}^{\text{Key-IND}}(\lambda)$

Game₁(λ): First, we modify the Corrupt Oracle to only return empty sets. This modification is undetectable by the adversary, as neither parties hold any state.

Game₂(λ): Now, we modify the Session-State Oracle to return empty sets. As with our first modification, this is undetectable, as the parties do not hold state.

Game₃(λ): Next, we modify the Reveal Oracle to always return truly random strings. This modification is undetectable by the adversary, as successful execution of the protocol yields a truly random session key. The exception to this is some failure in the QKD system, which results in non-uniform keys being output. This happens exactly with $\Pr[\text{QKD_fail}]$, which is exactly the success probability of a distinguisher $\mathcal{D}$ distinguishing Game 3 and Game 2.

Game₄(λ): Finally, we modify the Test Oracle to always return random keys. As with our previous modification to the Reveal Oracle, this is undetectable to the adversary because real keys are truly random.

Since the game's behavior is now completely independent of the secret bit b , the best strategy for the adversary is to guess. In particular, due to the modification of the Corrupt Oracle, forward secrecy is also achieved. As a result, Π_{QKD} achieves Key-IND up to $\Pr[\text{QKD_fail}]$.

The second requirement for a protocol to be SK-secure is that successfully completing the protocol must yield identical session keys for both parties. This is guaranteed by the underlying QKD system up to a likelihood of $\Pr[\text{QKD_fail}]$.

The requirement combination yields SK security up to the claimed bound. $\square$

Remark 1. Lemma 1 gives a security bound for any unbounded adversary. We note that this directly implies security against the strictly weaker class of QPT adversaries, which we will use in Theorem 1.

We also note that due to the lack of classical long-term identity keys in QKD systems, they are, by definition, resistant to key compromise impersonation (KCI) attacks. Specifically, within our black-box model, the QKD sub-protocol does not expose an extractable long-term secret to the adversary.

2.7 Hybrid AKE

By Hybrid Authenticated Key Exchange (HAKE) we refer to utilizing multiple key exchange protocols simultaneously to achieve enhanced security even in the event that all except one of the component algorithms is compromised. We propose a direct construction for a hybrid authenticated key exchange based on KEMs and QKD, combining the resulting keys via a KDF following the paradigm proposed by BSI [18]. For our analysis, we adhere to the established CK01 model rather than the modified Bellare-Rogaway-style security model introduced by Dowling et al. [16].

3 The KEM-based HAKE protocol

A two-step approach is followed in the construction of the HAKE:

1. Execution of two different key exchanges. To achieve the desired hybridization properties, different hardness assumptions are used. (In this case QKD and regular cryptography.)
2. Key combination by means of a combiner function maintaining security as long as one of the component keys is secure, as described in Section 2.4.

These substeps can be carried out either sequentially or in parallel. In our construction, the QKD messages are sent in parallel with the PQ key exchange of [8], as illustrated in Figure 3. Two preconditions must be taken into account:

1. Both, Alice and Bob must maintain a permanent key pair (sk_a, pk_a) and (sk_b, pk_b) that is tied to their respective identity. They also know the public key of the intended communication partner.
2. Both parties have to have access to QKD nodes that directly communicate with each other by means of an appropriate physical connection for executing the underlying QKD protocol. The keys are obtained via a well-defined interface, as previously outlined in Definition 7.

3.1 Protocol Flow

Figure 3 illustrates the full protocol flow for our newly proposed PQC-QKD HAKE protocol Π_{HAKE} . It consists of two independent key exchanges, with the

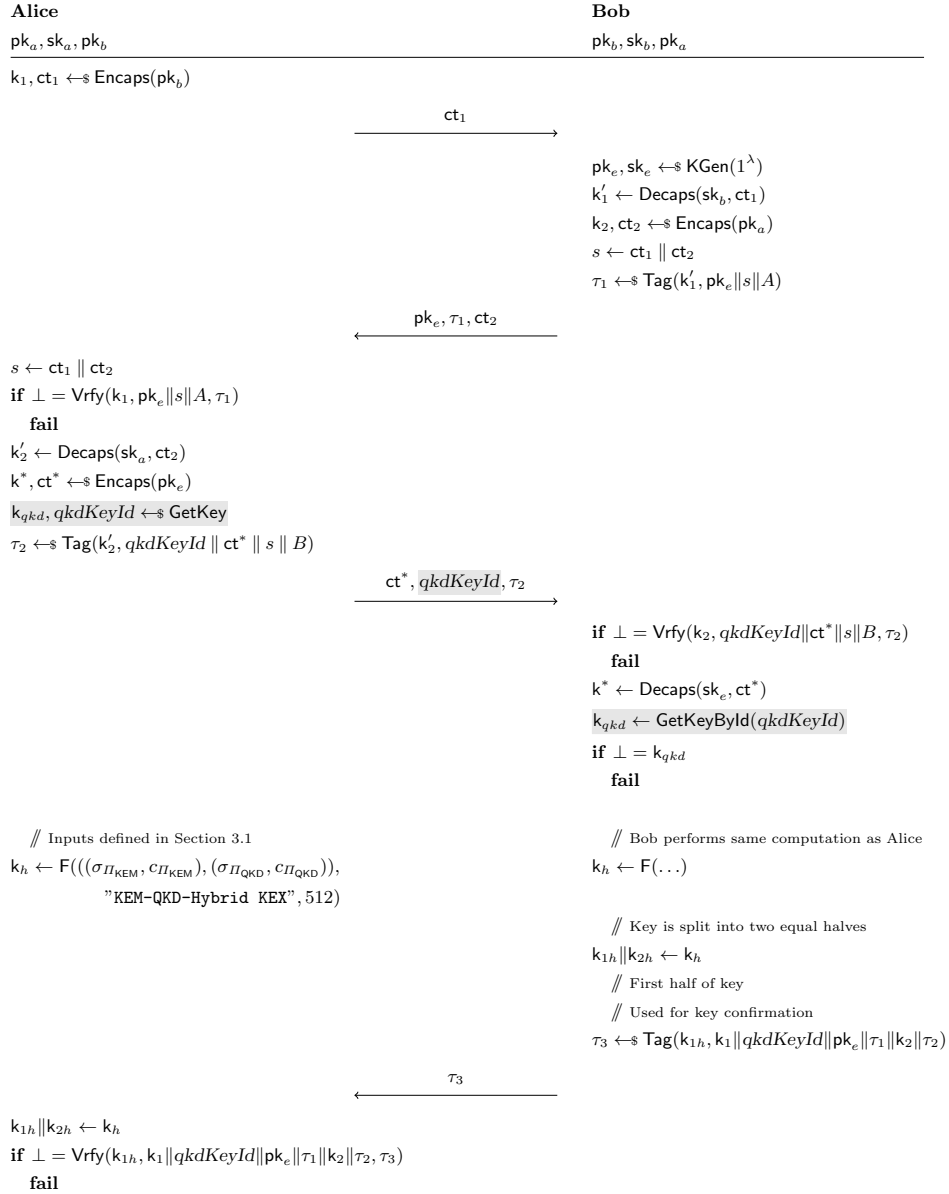


Fig. 3. Our proposed HAKE protocol Π_{HAKE} between Alice and Bob combining a KEM-based and a QKD key agreement. k_{2h} is the final 256-bit key shared between both parties. It is assumed that the parties own a public KEM key pk_a and pk_b , respectively, of each other. The QKD part of the protocol is composed of the **highlighted** actions. Note that the KEM and QKD sub-protocols operate independently (cf. Lemma 2); the second message from Alice to Bob batches transmissions from both sub-protocols to reduce round trips.

bulk of the messages being attributed to the scheme originally proposed by Boyd et al. [8] which realizes the desired functionality using generic KEMs as its only building block. The QKD part of the protocol requires fewer messages, which are highlighted in the protocol.

From a high-level perspective, Alice and Bob first use their long-term public keys to authenticate each other. To achieve forward secrecy, Bob generates a new ephemeral KEM keypair rather than using his long-term key directly; otherwise, compromise of the static secret key would reveal all past session keys. In parallel, they use their QKD access to establish a second shared key. Initially, Alice requests a fresh key from her KMS using `GetKey` and receives the key and its ID `qkdKeyId`. The ID is then transmitted to Bob, who uses it to retrieve the same key from his respective KMS. To reduce the number of messages, the ID is transmitted in parallel with a message from the KEM based AKE protocol. In general, the establishment of QKD keys necessitates a significantly greater investment of time and computing power than is required for KEM operations [33,30,9]. Furthermore, once a key has been requested at the QKD interface, it cannot be used a second time, as outlined in Definition 7. Consequently, the QKD key requests are executed in the latter phase of the protocol. In the event of an issue, such as a failure in the KEM-based authentication, no QKD key is wasted. Following the conclusion of both key exchanges, both parties run a key combiner in order to obtain the final hybrid key.

As explained in Section 2.4, our combiner design is based on the research by Backendal et al. and one of their proposed constructions based on SHA-3 [1, Theorem 8]. We will refer to the KDF as F . For evaluating (and later analyzing) the combiner-KDF in Backendal et al.’s security model, we need to define sources of key material (and the resulting secrets and contexts) as well as a label for the final execution of the combiner.

KEM-based AKE: For the KEM-based AKE protocol Π_{KEM} , the secret $\sigma_{\Pi_{\text{KEM}}}$ is the key resulting from the final decapsulation step and the context $c_{\Pi_{\text{KEM}}}$ is composed of all public keys, the transcript of the KEM-based sub-protocol and the identities Alice (A) and Bob (B):

$$\begin{aligned}\sigma_{\Pi_{\text{KEM}}} &:= k^* \\ c_{\Pi_{\text{KEM}}} &:= A \parallel \text{pk}_a \parallel B \parallel \text{pk}_b \parallel \text{pk}_e \parallel k_1 \parallel k_2 \parallel k^*\end{aligned}$$

QKD-based AKE: Due to the characteristics of QKD, it is much simpler to identify secret key material and context for the QKD-based AKE (Π_{QKD}):

$$\begin{aligned}\sigma_{\Pi_{\text{QKD}}} &:= k_{qkd} \\ c_{\Pi_{\text{QKD}}} &:= A \parallel B \parallel qkdKeyId\end{aligned}$$

The label `lbl` is a fixed, protocol-specific string that provides domain separation, ensuring that keys derived for different protocols or purposes cannot be confused; for our implementation, the string `KEM-QKD-Hybrid KEX` is used. The output

length is chosen to be 512 bits, leaving both parties with 256 usable bits after the key confirmation step.

The aforementioned key confirmation step occurs after the key combiner is executed and ensures that both parties have derived identical keys. It is important not to use the final key for this step, as this might leak information related to this final key. Instead, the output of the combiner, k_h is split into two keys of equal length, k_{1h} and k_{2h} . k_{1h} will be used for key confirmation, while k_{2h} is the resulting key of the protocol. The key confirmation itself is carried out using a MAC that authenticates the entire transcript of the conversation. The tag is created by Bob and verified by Alice. Note that the key confirmation is one-directional (Bob to Alice). Full mutual confirmation would require an additional message, but cannot provide both parties with equal assurance due to the well-known impossibility result for coordinated agreement [10]. In practice, the first message Alice sends using the derived key k_{2h} serves as implicit confirmation to Bob. The long-term public keys assumed above can be distributed by direct pre-distribution in small networks, via a QKDN controller acting as a trusted third party, or via a PKI [9,3]; our “without signatures” property concerns the online protocol execution, so any such bootstrapping—which incurs signature operations only as a one-time setup cost—is fully compatible with our construction.

4 Security

We now give the formal security statements for the HAKE protocol introduced in the previous sections. Intuitively, the security of the protocol follows from the security of either the KEM-based AKE or the security of the QKD system, as long as one of the component keys remains secure.

The two component key exchanges based on KEMs and QKD are independent, so their security can be proven individually. The hybrid guarantee is then established via a key combiner that binds both transcripts to the final key. We start by establishing this independence, which is the foundation for the modular analysis.

Lemma 2 (Independence of sub-protocols). *Π_{HAKE} as proposed in Figure 3 is composed of the KEM based AKE protocol Π_{KEM} , the QKD-based AKE protocol Π_{QKD} and a final combiner step using F . The two sub-protocols process their messages independently: neither sub-protocol’s computation depends on secrets or intermediate values of the other.*

Proof. The messages of Π_{KEM} and Π_{QKD} are processed separately throughout the protocol. The only shared message is the second transmission from Alice to Bob, which batches messages from both sub-protocols into one network packet. This is a transport-level optimization that does not create a cryptographic dependency, as the message can equivalently be split into two separate transmissions without affecting the security reduction. $\square$

Next, we establish the SK security of the two component protocols in the CK01 model.

Lemma 3 (SK security of the KEM based AKE). *Let $\mathcal{A}$ be an adversary against SK security. Observe that Π_{HAKE} as proposed in Figure 3 without the gray boxes representing the QKD part and the combiner actions at the end is an instance of a KEM-based authenticated key exchange as proposed and proven secure by Boyd et al. [8, Figure 8]. Since these extra messages are handled independently, they do not affect the security of the key exchange, which is shown to be SK-secure by Boyd et al. [8, Theorem 2,3,4] with the following bound for efficient QPT adversaries $\mathcal{B}$, $\mathcal{D}$, and $\mathcal{F}$:*

$$\text{Adv}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{SK-secure}}(\lambda) \leq \ell \cdot \left(\text{Adv}_{\text{KEM}, \mathcal{D}}^{\text{CCA}}(\lambda) + \text{Adv}_{\text{MAC}, \mathcal{F}}^{\text{EUF-CMA}}(\lambda) \right) + \ell \cdot \text{Adv}_{\text{KEM}, \mathcal{B}}^{\text{CPA}}(\lambda),$$

where $\ell := n^2q$, n denotes the number of parties, and q the number of challenge messages. The adversaries are as defined in [8].

Proof Outline. The proof composes Theorems 2, 3, and 4 of [8]; we refer to the original work for full details and highlight the main steps here.

Following the approach of [10], the protocol is first proven secure in the authenticated links model, where adversaries cannot tamper with messages. Then, using a KEM-based authenticator, the protocol is lifted to the real-world setting of unauthenticated links. The resulting bound is additive, equal to the sum of the bounds for the protocol itself and the authenticator.

In the stated bound, the IND-CCA security of the KEM and EUF-CMA of the MAC scheme are due to the authenticator, while the IND-CPA security of the underlying KEM is required by the key exchange in the authenticated links model. Adversaries $\mathcal{B}$, $\mathcal{D}$, $\mathcal{F}$ are as defined in [8, Proof of Theorem 3,4]. The multiplicative factor ℓ accounts for the probability that the inner adversary in the reduction selects the target session in which the challenge is embedded. $\square$

Theorem 1 (SK security of Π_{HAKE}). *Let Π_{HAKE} be the protocol introduced in Figure 3. A QPT adversary $\mathcal{A}$ attacking the scheme has its success probability bound by the term*

$$\text{Adv}_{\Pi_{\text{HAKE}}, \mathcal{A}}^{\text{SK-secure}}(\lambda) \leq 2 \cdot \min \left(\ell \cdot \left(\text{Adv}_{\text{KEM}, \mathcal{D}}^{\text{CCA}}(\lambda) + \text{Adv}_{\text{MAC}, \mathcal{F}}^{\text{EUF-CMA}}(\lambda) \right) + \ell \cdot \text{Adv}_{\text{KEM}, \mathcal{B}}^{\text{CPA}}(\lambda), \Pr[\text{QKD_fail}] \right),$$

where $\ell := n^2q$, n denotes the number of parties and q the number of challenge queries and $\mathcal{B}$, $\mathcal{D}$, $\mathcal{F}$ are the QPT adversaries as defined in Lemma 3.

Proof. The proof is composed of four components:

1. Independence of Π_{KEM} and Π_{QKD} (Lemma 2)

2. SK security of the KEM based AKE protocol (Lemma 3)
3. SK security of the QKD-based AKE protocol (Lemma 1)
4. Combiner Security of the KDF F (This proof and [1, Thm. 9])

In combination, the first three components, with the proofs stated in the associated lemmas above, allow us to focus on the security of the combiner function F for this proof. We have established that the two component-protocols are secure by themselves, it remains to prove that our combiner function F is a good combiner function that maintains SK security of the hybrid protocol Π_{HAKE} against QPT adversaries, as long as one of its components remains secure.

We show this result by following the work from Backendal et al. [1, Theorem 9]. Recall that our protocol uses the so-called RO – KDF, instantiated with SHA-3 with a fixed output length of 512 bits, as described in Section 2.4. SHA-3 is based on the Keccak sponge construction, which is indistinguishable from a random oracle [6], justifying the random oracle modeling used in Theorem 9 of [1]. Our instantiation is a 2-KDF with the source collection Σ containing the two sources Σ_{KEM} and Σ_{QKD} . The Σ -map maps Σ_{KEM} to the first input and Σ_{QKD} to the second input. Domain separation is achieved through the protocol-specific label lbl and the inclusion of both sub-protocol transcripts in the context, preventing cross-protocol attacks. Since we are building a QKD/PQ hybrid, we use the predicate $\text{req}_{\text{Hybrid}}$ to determine which queries the adversary is allowed to make. SHA-3 is a non-extendable output function (i.e. queries with the same values, but different output lengths do not return values that are prefixes of each other), also called NOF. Therefore, the freshness predicate by Backendal et al. for this type of output behavior req_N also applies here.

This exact setup was analyzed by Backendal et al., allowing us to directly apply [1, Theorem 9], yielding the following bound for an adversary $\mathcal{G}$ attacking the combiner function F with QPT adversaries $\mathcal{C}$ and $\mathcal{E}$:

$$\text{Adv}_{F, \mathcal{A}, \Sigma, \text{req}_N \wedge \text{req}_{\text{Hybrid}}}^{\text{kdf}}(\lambda) \leq 2 \cdot \min(\text{Adv}_{F, \mathcal{C}, \Sigma_{\text{KEM}}}^{\text{up}}(\lambda), \text{Adv}_{F, \mathcal{E}, \Sigma_{\text{QKD}}}^{\text{up}}(\lambda)).$$

This establishes combiner security: F yields a secure key provided that either source is unpredictable.

It remains to connect this to AKE security of the full protocol Π_{HAKE} . The reduction proceeds as follows: SK security of Π_{KEM} (Lemma 3) and Π_{QKD} (Lemma 1) implies pseudorandomness of the respective session keys, which in turn implies unpredictability [1, Lemma 23]. Substituting the SK security bounds of the two component schemes into the above inequality yields the claimed bound for Π_{HAKE} . $\square$

The $\min$ term in the final bound is exactly what we aim to achieve and gives us the desired property of the protocol being as secure as its best component.

While Theorem 1 establishes security in the well-established CK01 model, two advanced security properties might also be of interest. The first is security against unknown key-share (UKS) attacks, as first described in [15]. In Π_{HAKE} ,

both parties include the identity of their intended partner as part of the context information into the final key derivation step. Consequently any confusion about the communication partner would yield different keys for both partners, causing the key confirmation step to fail.

The second property is security against key compromise impersonation (KCI) attacks. Such attacks are possible if the final key depends only on the long-term key of one of the parties. However, running Π_{HAKE} includes calculating two MAC tags, one with a key resulting from an encapsulation under Alice’s identity key and the other from encapsulating under Bob’s identity key. An adversary who has compromised one party’s long-term key can compute the corresponding MAC tag, but cannot forge the tag derived from the uncompromised party’s key without breaking the IND-CCA security of the KEM. In combination with our earlier observation that the sub-protocol Π_{QKD} is also resistant against such attacks, we obtain security against KCI-style attacks for the resulting protocol Π_{HAKE} .

Conditional ITS Security. The $\min()$ expression in the security bound of Theorem 1 reveals an interesting subtlety: if the QKD system is working as intended, the hybridization does not degrade security to be resistant “only” against QPT adversaries, but maintains security against unbounded adversaries. This covers both conditions of SK security (Definition 6): key agreement and key indistinguishability. Authentication of the classical channel is an integral part of the QKD protocol and is subsumed by our black-box model, so an uncompromised QKD sub-protocol independently provides both guarantees against unbounded adversaries. This improvement is obtained by using the modern framework for analyzing KDF security by Backendal et al. [1], which allows for a fine-grained analysis of KDFs with multiple inputs. That the resulting proof of Theorem 2 is short reflects the power of this modular decomposition: the technical work resides in establishing the min-bound of Theorem 1 and the QKD guarantee of Lemma 1, after which the ITS conclusion follows directly. The contribution is not a complex proof technique, but the identification that the combination of a suitable combiner framework with QKD yields a guarantee that no purely classical or PQC-only hybrid protocol can provide.

We note that this result relies on the random oracle assumption used in [1, Theorem 9], which our protocol instantiates with SHA-3. Precisely, Theorem 2 establishes information-theoretic security *in the random oracle model*: the only idealization is the hash function, while the bound itself holds for computationally unbounded adversaries. This is qualitatively different from computational security guarantees that rely on the hardness of lattice or code-based problems, since the random oracle models a single, fixed-length hash evaluation rather than an asymptotic hardness assumption. A combiner achieving the same guarantee without the random oracle exists [1, Theorem 17] but lacks a standardized implementation.

Theorem 2 (Information theoretic security of Π_{HAKE}). *Let Π_{HAKE} be the protocol introduced in Figure 3. An unbounded adversary $\mathcal{A}$ attacking the scheme*

has its success probability bound by the term

$$\text{Adv}_{\Pi_{\text{HAKE}}, \mathcal{A}}^{\text{SK-secure}}(\lambda) \leq 2 \cdot \Pr[\text{QKD_fail}].$$

This is a significant improvement over using the HKDF scheme as suggested by NIST [2], which also yields hybrid guarantees, but only in a computational setting [29, Theorem 4], since its security proof inherently relies on the pseudo-randomness of HMAC.

Proof. By Theorem 1, the advantage of any adversary against Π_{HAKE} is bounded by $2 \cdot \min(\cdot, \Pr[\text{QKD_fail}])$. The first argument of the min depends on computational assumptions and is therefore only meaningful for QPT adversaries. However, the second argument, $\Pr[\text{QKD_fail}]$, is established in Lemma 1 without any computational assumption and holds for unbounded adversaries. Since the min is at most its smallest argument, the bound $2 \cdot \Pr[\text{QKD_fail}]$ applies regardless of the adversary’s computational power, yielding the claimed result. $\square$

In summary, Π_{HAKE} achieves a best-of-both-worlds guarantee: as long as the QKD component remains uncompromised, the protocol achieves full SK security against unbounded adversaries, even if all computational assumptions underlying the KEM-based AKE are broken. This strictly exceeds what classical-PQC hybrid protocols can offer and highlights the advantage of integrating QKD into a modular hybrid design. Conversely, if the QKD system is compromised, the KEM-based component ensures that the protocol remains secure against QPT adversaries under standard cryptographic assumptions, preserving the hybrid safety net.

5 Implementation and Evaluation

Based on the aforementioned protocol, a Hybridization Service (HS) was implemented. The source code is freely accessible and can be found at our Gitlab Repository³. HS has been written in the *Go* programming language and is run within a Docker container. It offers two interfaces, S_1 and S_2 . S_1 is employed for the purposes of initial communication and control by a higher-level application. It facilitates the transmission of a hybridization request, encompassing the address data of the opposing HS, in addition to the QKD key to be hybridized and its id. HS is also capable of acquiring a specific QKD key via S_1 using the QKD key id. S_2 is employed to facilitate the exchange of a PQC key with another HS instance, based on an initially configured KEM. The implementation of the PQC schemes is achieved through the utilization of liboqs [35] and the liboqs Go wrapper [34]. The *Go* crypto package [43] provides functionality for the implementation of traditional cryptographic mechanisms. The transmission of data is facilitated by the use of Transmission Control Protocol (TCP).

³ <https://code.fbi.h-da.de/hybridizekeyservice/gohybridizekeyservice>

5.1 Benchmarking Setup

The evaluation was conducted using a server configured with Ubuntu 24.04 LTS, 8 vCPUs (AMD EPYC Milan (DDR4-3200 ECC), 2.25 GHz), 16 GB RAM. Two Docker containers were executed, each containing an HS instance, designated HS Alice and HS Bob, respectively. Furthermore, two additional Docker containers each contain a KMS instance, designated KMS Alice and KMS Bob, respectively. HS Alice and HS Bob are assigned to their respective KMS and connected to them via S_1 . In addition, the HS instances are connected to each other via S_2 .

The evaluation encompassed the entire protocol sequence, commencing from the hybridization request via S_1 , culminating in the key receipt. The public and private keys necessary for KEM-based authentication were generated beforehand. Alice possesses its own key pair, (pk_a, sk_a) , as well as Bob's pk_b , stored within its internal memory. In the case of Bob the situation is reversed. Furthermore, QKD keys have already been established. The hybridization process can only be started once KMS Alice has a QKD key to hybridize and an associated ID, meaning that cases where QKD key establishment has not yet taken place are disregarded and part of KMS key management. Consequently, it is not necessary to account for additional delays in the QKD key establishment process. The pertinent standards and recommendations [26,27,38] also propose the implementation of KMSs with designated storage for QKDN nodes, thereby ensuring the establishment of QKD keys in advance and their maintenance in a state of readiness, as described in [14]. An empty key storage is an undesirable special case. Any delay caused by this will have an accumulating effect on the total duration of the hybridization request.

Each experiment consisted of 1000 measurements per configuration. A total of three different KEMs were tested, each with security levels 1, 3, and 5. ML-KEM (*ML-KEM-512*, *ML-KEM-768*, *ML-KEM-1024*) that has been standardized by National Institute of Standards and Technology (NIST), as well as FrodoKEM (*FrodoKEM-640-AES*, *FrodoKEM-976-AES*, *FrodoKEM-1344-AES*) and Classic McEliece (*Classic-McEliece-348864f*, *Classic-McEliece-460896f*, *Classic-McEliece-6688128f*), which were recommended by other institutions, such as the German BSI [20,18]. MAC was implemented using Hash-based MAC (HMAC) and SHA-256. For the key combination, SHA3-512 was used.

5.2 Experimental Results

The results for the simple test configuration are presented in Figure 4 in the form of a box plot diagram [36]. It is noteworthy that the ML-KEM variants exhibited an average time requirement of 11 to 13 ms with minimal dispersion. FrodoKEM is slightly slower, and the time penalties for the higher security levels are somewhat greater. In contrast, the variants of Classic McEliece are distinctly different. As demonstrated in Figure 4, the median values range from approximately 50 ms for security level 1, to 113 ms for security level 3, and 150 ms for security level 5. Since no network delays were emulated in this initial setup, the results are dominated by the computational cost of the asymmetric schemes.

ML-KEM is highly efficient, whereas Classic McEliece exhibits significant latency due to its key generation process (mceliece348864f 15-20 ms, mceliece6688128f up to 150 ms [12]). In the case of mceliece6688128f, this delay can reach up to 150 ms [11,12]. For each KEM, the minimum, 0.25 quantile, and median are close, indicating that most runs are near optimal.

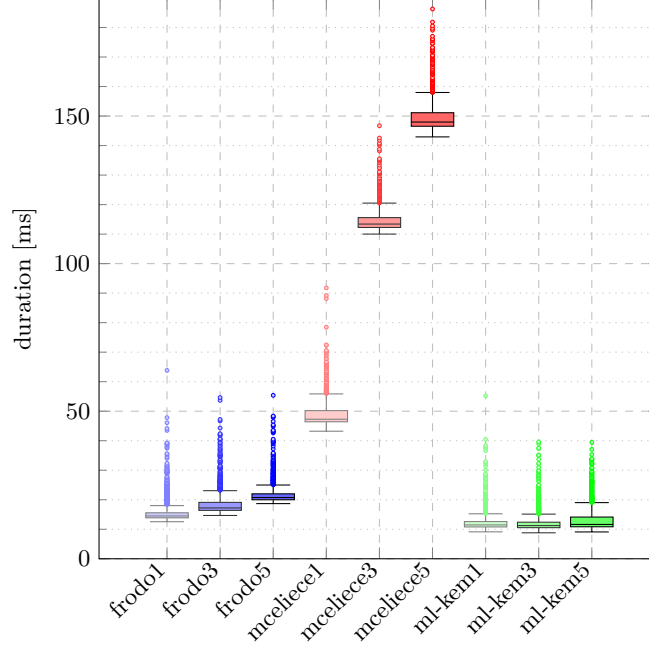


Fig. 4. Hybridization duration when using the PQC KEMs FrodoKEM (*FrodoKEM-640-AES*, *FrodoKEM-976-AES*, *FrodoKEM-1344-AES*), Classic McEliece (*Classic-McEliece-348864f*, *Classic-McEliece-460896f*, *Classic-McEliece-6688128f*), and ML-KEM (*ML-KEM-512*, *ML-KEM-768*, *ML-KEM-1024*). A total of 1,000 measurements were conducted for each of the three security levels (1, 3, and 5). Box plot diagram, which shows median, 0.25 quartile, 0.75 quartile, minimum, maximum, and outliers.

In a subsequent series of tests, the delay between each message transmitted between the HS instances was incremented by 20 ms. On average, the duration increases by 100 ms with the same KEM scheme when latency is increased by 20 ms, as demonstrated in Table 1. The protocol requires five sequential message transmissions (cf. Figure 3), so the observed increase of approximately $5 \times 20 \text{ ms} = 100 \text{ ms}$ matches the expected behavior. However, the amount of data to be transmitted becomes a salient factor, given that large messages are divided into multiple packets and an additional sequential transmission introduces an extra delay. It is important to note that the experiments did not consider additional network constraints, such as diminished bandwidth, packet loss, or

flow and congestion control under TCP. For a more comprehensive overview, please refer to the works of Paquin et al. [37] and Henrich et al. [23]. In particular, FrodoKEM and Classic McEliece possess significantly large key sizes, which would further reduce throughput under a limited TCP congestion window or constrained bandwidth.

Duration	0ms Latency [ms]	20ms Latency [ms]	Difference [ms]
Frodo1	14.503868	115.600346	101.096478
Frodo3	17.226027	117.882171	100.656144
Frodo5	20.746900	121.583129	100.836229
McEliece1	47.279915	147.685722	100.405807
McEliece3	113.400480	213.590071	100.189591
McEliece5	147.971071	248.159086	100.188015
ML-KEM1	11.472463	112.238871	100.766408
ML-KEM3	11.239579	112.292954	101.053375
ML-KEM5	11.630536	112.322540	100.692004

Table 1. Median measured hybridization time for different KEM schemes. Columns illustrate measurements for direct connection (0 ms) and with added latency of 20 ms, as well as the difference between them.

Since QKD modules are limited to distances of 100–150 km, message delays beyond 20 ms are unlikely in typical deployments [31,45], and the evaluation thus covers the relevant use case. Even in the worst case (Classic McEliece at security level 5 with 20 ms latency), the total hybridization time remains below 250 ms, which is acceptable for applications where key establishment can be performed ahead of time.

6 Conclusion and Future Work

We have presented a hybrid authenticated key exchange protocol that combines PQC and QKD without requiring digital signature schemes during protocol execution. By applying the multi-input KDF framework of Backendal et al. [1], we obtain a modular security proof in the CK01 model with a transparent min-bound (Theorem 1), and show that conditional information-theoretic security is achieved when the QKD component is uncompromised (Theorem 2). Our benchmarks confirm that the protocol is practical, with ML-KEM-based instantiations completing in under 15 ms and remaining well within acceptable latency bounds even under realistic network delays.

The elimination of signature schemes reduces the overall attack surface and system complexity, including the administrative burden of maintaining a crypto-inventory. However, the realization of these advantages is contingent on the spe-

cific cryptographic schemes employed and the fundamental requirements of the system in question.

Several directions for future work remain. The random oracle assumption underlying the ITS guarantee can be removed by adopting the non-RO combiner of [1, Theorem 17] once a standardized implementation becomes available. Extending the formal analysis to a composable security framework would allow reasoning about the protocol’s use as a building block within larger systems. Finally, a more fine-grained integration of the QKD model, capturing aspects such as key synchronization and rate limitations, would further strengthen the practical relevance of the security guarantees.

References

1. Backendal, M., Clermont, S., Fischlin, M., Günther, F.: Key derivation functions without a grain of salt. In: Fehr, S., Fouque, P.A. (eds.) EUROCRYPT 2025, Part VIII. LNCS, vol. 15608, pp. 393–426. Springer, Cham, Switzerland, Madrid, Spain (May 4–8, 2025). https://doi.org/10.1007/978-3-031-91101-9_14
2. Barker, E., Chen, L., (NIST), R.D.: Recommendation for key-derivation methods in key-establishment schemes. Tech. Rep. NIST Special Publication (SP) 800-56C-r2, National Institute of Standards and Technology (NIST), Gaithersburg, MD (Aug 2020). <https://doi.org/10.6028/NIST.SP.800-56Cr2>
3. Battarbee, C., Striecks, C., Perret, L., Ramacher, S., Verhaeghe, K.: Quantum-safe hybrid key exchanges with KEM-based authentication. EPJ Quantum Technology **12**(1), 128 (Dec 2025). <https://doi.org/10.1140/epjqt/s40507-025-00425-3>, <https://epjquantumtechnology.springeropen.com/articles/10.1140/epjqt/s40507-025-00425-3>
4. Bennett, C.H., Brassard, G.: Quantum cryptography: Public key distribution and coin tossing. Theoretical Computer Science **560**, 7–11 (Dec 2014). <https://doi.org/10.1016/j.tcs.2014.05.025>, <http://dx.doi.org/10.1016/j.tcs.2014.05.025>
5. Bennett, C.H., Brassard, G., Mermin, N.D.: Quantum cryptography without bell’s theorem. Phys. Rev. Lett. **68**, 557–559 (Feb 1992). <https://doi.org/10.1103/PhysRevLett.68.557>, <https://link.aps.org/doi/10.1103/PhysRevLett.68.557>
6. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: On the indistinguishability of the Sponge construction. In: Smart, N.P. (ed.) EUROCRYPT 2008. LNCS, vol. 4965, pp. 181–197. Springer Berlin Heidelberg, Germany, Istanbul, Turkey (Apr 13–17, 2008). https://doi.org/10.1007/978-3-540-78967-3_11
7. Bindel, N., Brendel, J., Fischlin, M., Goncalves, B., Stebila, D.: Hybrid key encapsulation mechanisms and authenticated key exchange. In: Ding, J., Steinwand, R. (eds.) PQCrypto 2019: Post-Quantum Cryptography. Lecture Notes in Computer Science, vol. 11505, pp. 206–226. Springer, Berlin, Heidelberg, DE (Juli 2019). https://doi.org/https://doi.org/10.1007/978-3-030-25510-7_12, veranstaltungstitel: 10th International Conference on Post-Quantum Cryptography
8. Boyd, C., de Kock, B., Millerjord, L.: Modular design of KEM-based authenticated key exchange. In: Simpson, L., Baee, M.A.R. (eds.) ACISP 23. LNCS, vol. 13915, pp. 553–579. Springer, Cham, Switzerland, Brisbane, QLD, Australia (Jul 5–7, 2023). https://doi.org/10.1007/978-3-031-35486-1_24

9. Bruckner, S., Ramacher, S., Striecks, C.: Muckle+: End-to-end hybrid authenticated key exchanges. In: Johansson, T., Smith-Tone, D. (eds.) *Post-Quantum Cryptography*. pp. 601–633. Springer Nature Switzerland, Cham (2023)
10. Canetti, R., Krawczyk, H.: Analysis of key-exchange protocols and their use for building secure channels. In: Pfitzmann, B. (ed.) *EUROCRYPT 2001*. LNCS, vol. 2045, pp. 453–474. Springer Berlin Heidelberg, Germany, Innsbruck, Austria (May 6–10, 2001). https://doi.org/10.1007/3-540-44987-6_28
11. Chen, M.S., Chou, T.: Classic McEliece on the ARM cortex-m4. *Cryptology ePrint Archive*, Paper 2021/492 (2021), <https://eprint.iacr.org/2021/492>
12. Chen, P.J., Chou, T., Deshpande, S., Lahr, N., Niederhagen, R., Szefer, J., Wang, W.: Complete and improved FPGA implementation of classic McEliece. *IACR TCHES* **2022**(3), 71–113 (2022). <https://doi.org/10.46586/tches.v2022.i3.71-113>
13. Cramer, R., Shoup, V.: Design and analysis of practical public-key encryption schemes secure against adaptive chosen ciphertext attack. *SIAM Journal on Computing* **33**(1), 167–226 (2003)
14. Dervisevic, E., Tankovic, A., Fazel, E., Kompella, R., Fazio, P., Voznak, M., Mehic, M.: Quantum key distribution networks - key management: A survey. *ACM Comput. Surv.* **57**(10) (May 2025). <https://doi.org/10.1145/3730575>, <https://doi.org/10.1145/3730575>
15. Diffie, W., van Oorschot, P.C., Wiener, M.J.: Authentication and authenticated key exchanges. *DCC* **2**(2), 107–125 (1992). <https://doi.org/10.1007/BF00124891>
16. Dowling, B., Hansen, T.B., Paterson, K.G.: Many a mickle makes a muckle: A framework for provably quantum-secure hybrid key exchange. In: Ding, J., Tillich, J.P. (eds.) *PQCrypto 2020*. pp. 483–502. Springer, Cham, Switzerland, Paris, France (Apr 15–17, 2020). https://doi.org/10.1007/978-3-030-44223-1_26
17. Dworkin, M.J.: Sha-3 standard: Permutation-based hash and extendable-output functions: (2015-07-01 04:07:00 2015). <https://doi.org/https://doi.org/10.6028/NIST.FIPS.202>
18. Ehlen, S., et al.: Kryptografie quantensicher gestalten. Tech. Rep. BSI-Bro21/01, Bundesamt für Sicherheit in der Informationstechnik (BSI) (2021)
19. Ekert, A.K.: Quantum cryptography based on bell’s theorem. *Phys. Rev. Lett.* **67**, 661–663 (Aug 1991). <https://doi.org/10.1103/PhysRevLett.67.661>, <https://link.aps.org/doi/10.1103/PhysRevLett.67.661>
20. French Cybersecurity Agency (ANSSI), Federal Office for Information Security (BSI), Netherlands National Communications Security Agency (NLNCSA), Swedish National Communications Security Authority: Position paper on quantum key distribution. Tech. Rep. 1.0, BSI (Jan 2024)
21. Fujioka, A., Suzuki, K., Xagawa, K., Yoneyama, K.: Strongly secure authenticated key exchange from factoring, codes, and lattices. In: Fischlin, M., Buchmann, J., Manulis, M. (eds.) *Public Key Cryptography – PKC 2012*. pp. 467–484. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
22. Giron, A., et al.: Post-quantum hybrid key exchange: a systematic mapping study. *Journal of Cryptographic Engineering* **13**, 71–88 (2022)
23. Henrich, J., et al.: Performance impact of pqc kems on tls 1.3 under varying network characteristics. In: *Information Security: 26th Int. Conf. (ISC)*. p. 267–287. Springer, Cham, Berlin, Heidelberg, DE (2023). https://doi.org/10.1007/978-3-031-49187-0_14
24. Hövelmanns, K., Planken, D., Schaffner, C., Verschoor, S.R.: Qkd oracles for authenticated key exchange (2025), <https://arxiv.org/abs/2509.12478>

25. Hülsing, A., Ning, K.C., Schwabe, P., Weber, F.J., Zimmermann, P.R.: Post-quantum wireguard. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 304–321. IEEE, Piscataway, NJ, US (2021). <https://doi.org/10.1109/SP40001.2021.00030>
26. International Telecommunication Union Telecommunication Standardization Sector: ITU-T Y.3802 Quantum key distribution networks - Functional architecture. Tech. rep., ITU-T (2020)
27. International Telecommunication Union Telecommunication Standardization Sector: ITU-T Y.3803 Quantum key distribution networks – Key management. Tech. rep., ITU-T (2020)
28. Krawczyk, H.: HMQV: A high-performance secure Diffie-Hellman protocol. In: Shoup, V. (ed.) CRYPTO 2005. LNCS, vol. 3621, pp. 546–566. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 14–18, 2005). https://doi.org/10.1007/11535218_33
29. Krawczyk, H.: Cryptographic extraction and key derivation: The HKDF scheme. In: Rabin, T. (ed.) CRYPTO 2010. LNCS, vol. 6223, pp. 631–648. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 15–19, 2010). https://doi.org/10.1007/978-3-642-14623-7_34
30. Li, J., et al.: Integration of quantum key distribution networks and classical networks: An evolution perspective. *IEEE Network* **39**(3), 180–187 (2025). <https://doi.org/10.1109/MNET.2025.3537691>
31. Martin, V., et al.: Madqci: a heterogeneous and scalable sdn-qkd network deployed in production facilities. *npj Quantum Information* **10**(1), 80 (2024)
32. Mayers, D.: Unconditional security in quantum cryptography. *J. ACM* **48**(3), 351–406 (May 2001). <https://doi.org/10.1145/382780.382781>, <https://doi.org/10.1145/382780.382781>
33. Mehic, M., et al.: Quantum key distribution: A networking perspective. *ACM Comput. Surv.* **53**(5), 1–41 (2020), <https://doi.org/10.1145/3402192>
34. Open Quantum Safe Project: liboqs-go: Go bindings for the open quantum safe library. <https://github.com/open-quantum-safe/liboqs-go> (2024), version 0.14.0, accessed 2026-02-04
35. Open Quantum Safe Project: liboqs: Open quantum safe cryptographic library. <https://github.com/open-quantum-safe/liboqs> (2024), version 0.14.0, accessed 2026-02-04
36. Páez, A., Boisjoly, G.: Exploratory Data Analysis, pp. 25–64. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-20719-8_2, https://doi.org/10.1007/978-3-031-20719-8_2
37. Paquin, C., Stebila, D., Tamvada, G.: Benchmarking post-quantum cryptography in tls. In: Ding, J., Tillich, J.P. (eds.) Post-Quantum Cryptography. pp. 72–91. Springer International Publishing, Cham (2020)
38. (QKD), E.I.S.G.I.Q.K.D.: Quantum Key Distribution (QKD), Application Interface. etsi gs qkd 014 v1.1.1 (2019)
39. Ricci, S., Dobias, P., Malina, L., Hajny, J., Jedlicka, P.: Hybrid keys in practice: Combining classical, quantum and post-quantum cryptography. *IEEE Access* **12**, 23206–23219 (2024). <https://doi.org/10.1109/ACCESS.2024.3364520>
40. Schanck, J., et al.: Criteria for selection of public-key cryptographic algorithms for quantum-safe hybrid cryptography. Tech. rep., IETF (2016), <https://datatracker.ietf.org/doc/html/draft-whyte-select-pkc-qsh-02>
41. Schwabe, P., Stebila, D., Wiggers, T.: Post-quantum TLS without handshake signatures. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer

- and Communications Security. p. 1461–1480. CCS '20, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3372297.3423350>
42. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.* **26**(5), 1484–1509 (Oct 1997). <https://doi.org/10.1137/S0097539795293172>
 43. The Go Authors: Go standard library: crypto package. <https://pkg.go.dev/crypto> (2025), version go1.25.0, accessed 2026-02-04
 44. Xu, G., Mao, J., Sakk, E., Wang, S.P.: An overview of quantum-safe approaches: Quantum key distribution and post-quantum cryptography. In: 2023 57th Annual Conference on Information Sciences and Systems (CISS). pp. 1–6. IEEE, Piscataway, NJ, US (2023). <https://doi.org/10.1109/CISS56502.2023.10089619>
 45. Zhong, X., Chen, L., Li, Y., Jin, G.: Design and verification of tcp/ip offload engine in quantum key distribution system. *Review of Scientific Instruments* **90**(11), 115102 (11 2019). <https://doi.org/10.1063/1.5117336>, <https://doi.org/10.1063/1.5117336>